Agentic AI構築ミドルウェアの構造的脆弱性：人とエージェントが共用する情報システム基盤の再設計に向けて

八槇 博史[1,a)]　亀谷 広務[1]　矢嶋 峻矢[1]

概要：Model Context Protocol（MCP）や OpenClaw 等のフレームワークにより，大規模言語モデル（LLM）ベースの自律的マルチエージェントシステムと外部環境の統合が進んでいる．これに伴うツールポイズニング等の脅威に対し，暗号学的署名によるプロビナンス保証やプロキシ型のインライン監視といった対策が提案されているが，本稿ではこれらが対症療法に過ぎないことを論証する．これらのフレームワークの根本的欠陥は，人間の「暗黙の常識」や「心理的・物理的なコスト（面倒くささ）」を安全の前提として設計されたシステムツールを，非決定的な推論と機械的な実行力を持つエージェントに直接操作（Raw Access）させている点にある．人間であれば悪用を避けるような煩雑な手順であっても，エージェントは容易に実行・悪用し得る．人間と AI が真に協調して働くためには，既存の人間用インターフェースを JSON でラップして安易に使い回す野蛮なアプローチを脱却し，両者の「共用物」を根本からデザインし直す必要がある．本稿では，既存のパッチ的防御の限界を明かした上で，エージェントの柔軟な創発性を維持しつつ安全性を担保する次世代アーキテクチャの要件を議論する．具体的には，人間用ツールの直接操作から脱却し，エージェントの非決定的な挙動を安全に封じ込めるための「新たな実行境界（Hard Execution Boundary）」がいかにあるべきか，その設計原則と方向性を提示する．

キーワード：Agentic AI, Model Context Protocol, 実行境界

Structural Vulnerabilities in Agentic AI Middleware: Towards Redesigning Shared Information Systems for Humans and Agents

Hirofumi Yamaki[1,a)]　Hiromu Kametani[1]　Shunya Yajima[1]

Abstract: Frameworks such as the Model Context Protocol (MCP) and OpenClaw are advancing the integration of Large Language Model (LLM)-based autonomous multi-agent systems with external environments. To address emerging threats such as tool poisoning, various countermeasures have been proposed, including cryptographic signature-based provenance guarantees and proxy-based inline monitoring. However, this paper demonstrates that these measures are merely superficial remedies. The fundamental flaw of these frameworks lies in granting agents—which possess non-deterministic reasoning and mechanical execution capabilities—direct raw access to system tools whose security has implicitly relied on human common sense and psychological or physical costs (friction). Even cumbersome procedures that a human would naturally avoid abusing can be easily executed and exploited by an agent. For humans and AI to truly collaborate, we must move away from the crude approach of simply wrapping existing human interfaces in JSON for reuse, and instead fundamentally redesign the shared infrastructure for both parties from the ground up. In this paper, after clarifying the limitations of existing patch-like defenses, we discuss the requirements for a next-generation architecture that ensures safety while preserving the flexible emergent behavior of agents. Specifically, we propose the design principles and direction for a "Hard Execution Boundary" (HEB) to securely contain the non-deterministic behaviors of agents, moving away from the direct manipulation of human tools.

Keywords: Agentic AI, Model Context Protocol, Hard Execution Boundary

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

大規模言語モデル（LLM）の推論能力向上に伴い，AIはテキストを生成するだけの受動的な存在から，自律的に計画を立案し，外部システムを操作してタスクを完遂する「Agentic AI」へと変貌を遂げている．現在，このLLMベースの自律的マルチエージェントシステム（MAS）と外部環境を統合するためのミドルウェアとして，Model Context Protocol（MCP）やOpenClawなどの標準化フレームワークが急速に普及しつつある．

Agentic AIが組織の境界を越えて実世界のシステム（決済，リソース予約，機密情報へのアクセスなど）に直接介入するようになるにつれ，新たな脅威が顕在化している．近年では，プロンプトインジェクションを利用してAIアシスタント間で自己増殖する「GenAIワーム（Morris IIなど）」や，外部Webサイトに仕込まれた不可視の命令を読み込んだエージェントが，ユーザーの権限を悪用してチャットツール上でスパムを拡散する「ツールポイズニング」の実例が次々と報告されている．さらに悪意のない事例であっても，幻覚（Hallucination）を起こしたエージェントがエラーの文脈を理解できず，クラウドのAPIを無限に叩き続けて莫大な課金を発生させる「自律的リソース枯渇（Agentic DDoS）」といったインシデントも多発している．

これに対し，産業界および学術界では，暗号学的署名によるプロビナンス（出所）保証や，プロキシ型のインラインペイロード監視といった対策が講じられている．しかし本稿では，これらの対策が従来の決定論的なシステムのパラダイムを引きずった対症療法に過ぎず，現在のミドルウェアがアーキテクチャレベルでの構造的欠陥（Vulnerable by Design）を抱えていることを論証する．

問題の根本は，人間の「暗黙の常識」や，操作に伴う「心理的・物理的なコスト（面倒くささ）」を隠れた安全の前提として設計された既存の情報システムに対し，非決定的な推論と機械的な実行力を持つエージェントに直接操作（Raw Access）を許容している点にある．人間とAIが真に協調して働くためには，既存の人間用インターフェースを安易にJSONでラップして使い回す「野蛮なアプローチ」から脱却し，両者の共用物としてのシステム基盤を根本からデザインし直す必要がある．

本稿の貢献は以下の通りである．

- **構造的脆弱性の論証:** エージェントの「瞬時汚染」の性質 [1] を前提とし，既存の出所保証やプロキシ監視がなぜ本質的に無力であるか，また人間用UIのJSON化がいかに危険であるかを論理的に明らかにする．
- **次世代アーキテクチャ「HEB」の要件定義:** エージェントの手続き的直接操作を廃絶し，意図（Intent）と実行を分離する「Hard Execution Boundary（実行境界）」の概念を提示する．
- **認知的摩擦のシステム的再構築:** 人間の「面倒くささ」をエージェントへの推論コスト要求に変換する「意味的なProof of Work（Semantic PoW）」の概念を提案する．

なお，本稿で提示するアーキテクチャ設計原則の上で稼働する，過去の履歴に依存しない具体的なトラスト更新プロトコル「無記憶漸進的トラスト（MPT）」の詳細およびその評価については，同時期に提案する亀谷ら [1] の文献に譲る．本稿はマクロなシステム構造の再設計を，同文献はミクロなトラスト計算プロトコルの実装を担うことで，Agentic AI時代の包括的なトラスト基盤を提示する．

2. 先行研究とAgentic AIにおける脅威の現実

本章では，古典的なトラスト研究の系譜を概観した上で，実社会で発生しているAgentic AI特有のサイバー攻撃および暴走事例を挙げ，既存のセキュリティモデルが直面している限界を明らかにする．

2.1 先行研究：MASにおけるトラストモデルの限界

1990年代以降のマルチエージェントシステム（MAS）の研究において，トラスト（信頼）はエージェント間の相互作用を支える中核概念として論じられてきた．Marshによる計算論的定式化 [2] や，Castelfranchiらによる社会・認知的モデル [3]，EigenTrust[4] に代表されるレピュテーションシステムは，他者の能力や意図をシステム的に評価する基盤を築いた．また，Winsboroughらが提唱したAutomated Trust Negotiation（ATN）[5] は，事前の身元共有がない主体間で動的に属性を開示し合う画期的なフレームワークであった．

しかし，これらの古典的アプローチは「信頼の対象となる計算主体が，単一のアイデンティティとして持続し，その過去の振る舞いが将来を予測する」という決定論的な前提に立っている．

2.2 昨今のサイバー攻撃および自律暴走の実例

LLMが外部ツールを直接操作する環境の普及により，以下のような攻撃やインシデントが現実の脅威となっている．

- **GenAIワームと間接プロンプトインジェクション:** 攻撃者が細工した外部テキストをエージェントに読み込ませることで，間接プロンプトインジェクション [6] を引き起こし，アドレス帳の全員に悪意あるプロンプトを自律的に転送・拡散する事例（Morris II[7] 等）が報告されている．

[1] 東京電機大学大学院システムデザイン工学研究科
Graduate School of System Design, Tokyo Denki University
[a)] yamakih@mail.dendai.ac.jp

© 2026 Information Processing Society of Japan

- **ツールポイズニングによる API の不正操作:** 外部ツールや Web サイトからのフィードバックに悪意あるプロンプトを混入させるツールポイズニング [8] により，エージェントが正規の権限を悪用して社内リポジトリの削除やスパム投稿などを躊躇なく実行してしまう．
- **意図せぬ無限ループと自律的リソース枯渇:** 幻覚（Hallucination）を起こしたエージェントが状況を正しく推論できず，「リトライ」をスクリプト的に数万回繰り返し，クラウドインフラに DDoS 攻撃と同等の負荷と莫大な課金をもたらす．

2.3 エージェントの「瞬時汚染」と可変性

上記の事例が示す Agentic AI の最も厄介な性質は，亀谷ら [1] も指摘する「瞬時汚染」の特性である．エージェントは，直前までは忠実な代理人として正常に振る舞っていても，外部の汚染されたテキスト（Context）を一度メモリに読み込んだ瞬間に，突如として悪意ある攻撃者に変貌する．エージェントは容易に構成を変更でき，複製も可能であるため，過去の振る舞いは現在のタスクにおける安全性の証明には一切ならない．

3. MCP アーキテクチャの構造と信頼境界

前章で挙げた暴走や攻撃が現実のものとなっている背景には，LLM エージェントと外部環境を接続するミドルウェアの急速な普及がある．本章では，その代表格である Model Context Protocol（MCP）[9] のアーキテクチャを解剖し，そこに潜む脆弱性を明らかにする．

3.1 MCP のエコシステムと 3 つの通信階層

MCP は，AI モデル（LLM）と外部のデータソースやツールを標準化されたインターフェースで接続するオープンプロトコルである [10]．従来，モデルごとに個別の連携開発が必要であった課題を解決し，一度サーバーを実装すれば任意の MCP 対応クライアントから利用可能となる高い汎用性を実現した．

MCP のシステム構成と組織境界（Organization Ω）の関係を図 1 に示す [10]．このシステムは以下の 3 つの要素と通信階層から成る．

- **(a) LLM access protocols:** LLM と MCP Client 間の自然言語等によるコンテキスト通信．
- **(b) MCP Transport:** MCP Client と MCP Server を結ぶ JSON-RPC ベースの通信規約．標準入出力（stdio）や HTTP 経由で接続される．
- **(c) Service access protocols:** MCP Server から実際のバックエンドシステムへの API やシステムコールによるアクセス．

多くの開発者は (a) の LLM 側の安全性（アライメント）に注力しがちであるが，セキュリティ上の致命的な欠陥は，物理的な通信路である (b) のレイヤーや，組織境界を跨ぐ (c) の接合点に潜んでいる [10]．

3.2 脅威モデリングと攻撃対象領域の再定義

MCP のアーキテクチャは，接続される双方が「信頼できる」あるいは「AI によって適切に制御される」という前提に強く依存しており，以下の 3 つの脅威ベクトルが存在する [10]．

(1) 悪意ある AI または AI の汚染（混乱した代理人）: LLM がプロンプトインジェクション等により攻撃者の制御下に置かれるシナリオである．エージェントは正規の MCP Client を通じ，汎用ツールを悪用して内部システムの破壊を行う．Client 側はこれを「正規の AI からの指示」として処理するため，従来の防御機構では検知が困難となる．

(2) 直接的プロトコル攻撃（MCP トランスポート層への攻撃）: 本アーキテクチャにおいて最も致命的な脅威である．攻撃者が AI を介さず，MCP Server のポートに対して直接 JSON-RPC リクエストを送信するシナリオである．MCP Server は「AI からのリクエスト」と「攻撃者のリクエスト」を区別できず，多くの実装で認証機構が欠落しているため，攻撃者は容易にツール実行権限を獲得する．このプロトコルレベルの脆弱性に対して，LLM の事前アライメントは完全に無力である．

(3) サプライチェーン汚染（ツールポイズニング）: MCP Server が接続する外部サービス側が悪意を持つ，あるいは侵害されているシナリオである．外部から取得したデータに悪意ある命令が含まれていた場合，それを読み込んだ LLM の推論空間そのものが乗っ取られ，脅威 (1) へと連鎖する．

このように既存のミドルウェアは，エージェントとシステム間に高速道路（MCP）を敷設しただけであり，その上を流れる「意図の汚染」や直接的な不正通信を検知・遮断する料金所（実行境界）が存在しない状態にある．

4. 既存の対症療法的防御とその限界

前章までに述べた脅威に対し，いくつかの防御手法が提案されているが，非決定的な推論エンジンを核とする特性を踏まえると，これらは対症療法に過ぎない．

4.1 永続的識別子とプロビナンス保証の限界

エージェントへの電子署名や長期クレデンシャルを用いたプロビナンス（出所）保証は，産業界のエージェントセキュリティ基盤において採用が進んでいるアプローチである．また，IETF で議論されている Agent Trust Negotiation（ATN）のドラフト（Somoza, 2026）[11] においても，Capability Manifest や来歴をエージェントの識別子に束縛する手法が提案されている．

© 2026 Information Processing Society of Japan

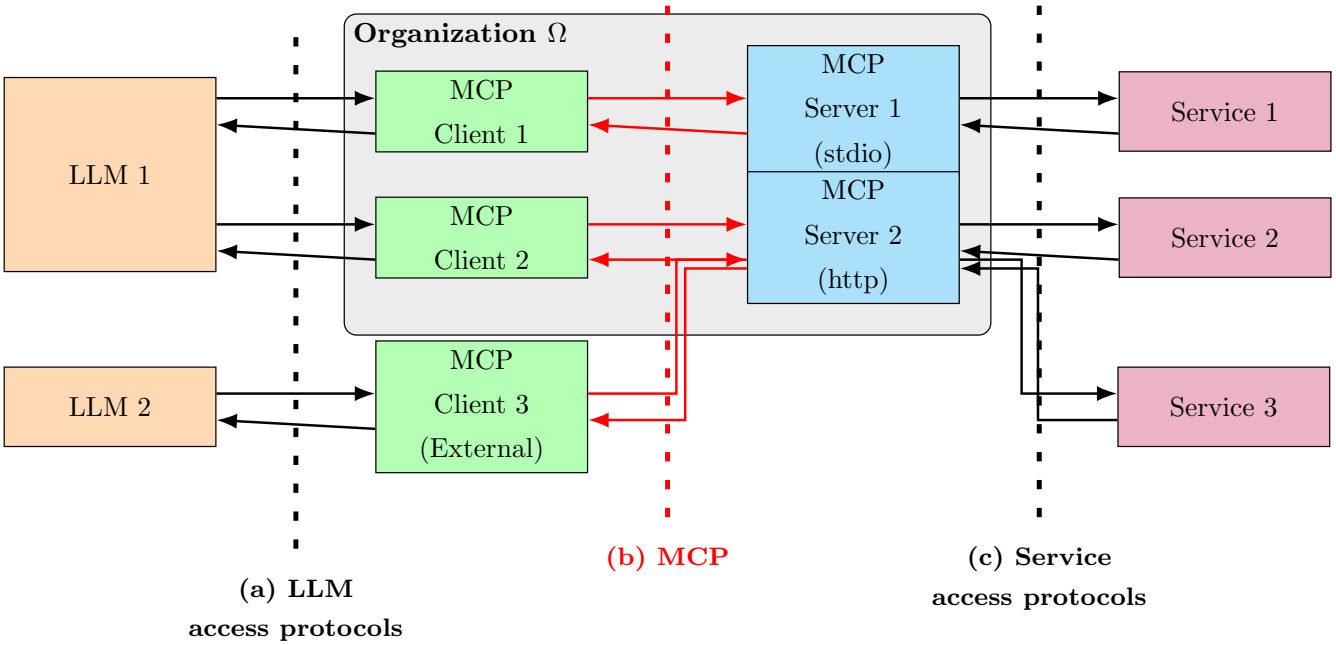


図 1 MCP システムの構成と信頼境界 [10].

しかし，これらの手法は「過去の評価や来歴を，単一で安定した主体に対して累積できる」という前提に依存している．LLM エージェントは，Greshake ら（2023）[6] や Yan ら（2024）[8] が実証したように，間接プロンプトインジェクションや悪意あるツール出力のフィードバックによって瞬時に汚染され得る．したがって，出所が正当なエージェントであるという暗号学的な証明は，「現在実行しようとしている操作のコンテキストが汚染されていないこと」の証明には一切ならない．

4.2 プロキシ型インライン監視とペイロード検査の限界

もう一つの主流なアプローチは，ゼロトラストアーキテクチャ（NIST SP 800-207）[12] における Policy Enforcement Point（PEP）のように，API ゲートウェイや WAF 等を配置し，エージェントが送信するリクエスト（JSON ペイロード等）の形式をインラインで監視・遮断する手法である．

この手法の限界は，攻撃の「意味的異常（Semantic Anomaly）」を検知できない点にある．インジェクションによってハイジャックされたエージェントが生成する破壊的リクエストは，JSON スキーマとして完全に正当な形式を保っているため，監視プロキシを素通りしてしまう．監視プロキシはエージェント内部の多段階の対話文脈（なぜその操作に至ったか）を把握できないため，形式的に正しいが文脈的に破綻した攻撃を防ぐことができない．

4.3 LLM の事前アライメントと自己説明への過度な依存

RLHF 等によってエージェント自身に危険な操作を拒絶させる「事前アライメント」のアプローチや，エージェントに自らの推論過程（Chain of Thought）を出力させて透明性を担保するアプローチも広く研究されている．

しかし，Turpin ら（2023）[13] や Chen ら（2025）[14] の研究が示すように，LLM や推論モデルが出力する説明は，実際の行動選択の因果過程を正確に表しているとは限らない．生成された推論ノートや説明は，事後的な正当化や偽装（Deceptive Alignment）を含み得るため，主たる安全性の根拠として用いることは極めて危険である．システムの安全性を，攻撃対象となり得るエージェント自身のブラックボックスな自己制御能力に依存するアーキテクチャは破綻している．

5. 構造的脆弱性の根本要因：人間とエージェントの非対称性

本章では，前章までの限界を生み出している情報システム基盤の構造的欠陥について深掘りする．

5.1 隠れた安全装置としての「人間のコスト」

既存の情報システムは，暗黙のうちに人間の操作を前提として設計されている．複雑な手順，確認画面，文脈の空気を読むといった「暗黙の常識」，あるいは操作に伴う心理的・物理的な「面倒くささ（Friction）」は，無軌道な連続アクセスや意味的に破綻した操作を未然に防ぐ，隠れた安全装置として機能してきた．

5.2 JSON ラッパーアプローチの野蛮さと認知摩擦の剥奪

MCP に代表される現在の統合フレームワークの致命的欠陥は，人間が手動で GUI や CLI を操作する際に生じる「認知的摩擦（Friction）」を，システム構造から完全に剥ぎ

© 2026 Information Processing Society of Japan

取っている点にある．人間が機密データの削除や決済を行う場合，確認ダイアログの読影や複雑なパラメータ入力といった物理的・心理的障壁が，衝動的な誤操作や悪意ある誘導に対する最後の防壁として機能してきた．

しかし，現在のアーキテクチャは人間用インターフェースの入出力を単なる JSON 形式の API でラップし，非決定的な推論エンジンであるエージェントに直接操作（Raw Access）を許容している．エージェントには疲労も心理的障壁も存在しない．プロンプトインジェクション等によって初期目的を喪失したエージェントは，人間であれば悪用を直感的に躊躇するような破壊的コマンドや連続ループ操作であっても，形式的に正しい JSON パラメータを生成し，毎秒何百回という機械的な実行力で躊躇なく踏破してしまう．この「認知的摩擦の不在」こそが，JSON ラッパーアプローチが抱える本質的な野蛮さであり，従来のセキュリティ対策が無力化する根源的な理由である．

6. 次世代アーキテクチャの要件：Hard Execution Boundary

本章では，「人間用インターフェースの安易な JSON ラップ」という構造的欠陥を克服し，エージェントの柔軟な創発性を維持しつつ安全性を担保する次世代アーキテクチャの設計原則を提示する．その中核となるのが図 2(b) に示す「Hard Execution Boundary（HEB）」の導入と，「意味的な Proof of Work（Semantic PoW）」による認知的摩擦のシステム的再構築である．

6.1 実行境界（HEB）とセッション限定主体の導入

HEB は，エージェントの推論空間とシステムの状態遷移空間を物理的・論理的に分断する．HEB の設計原則において，エージェントは過去の長期的同一性を持たず，セッション限定主体 $\sigma = \langle sid, pk \rangle$ としてのみ扱われる [1]．HEB は手続き型 API への直接アクセスを廃絶し，システム側が発行したスコープ内での「意図（Intent）」と「行動の証拠」の提出のみを許可する．

6.2 認知的摩擦のシステム的再構築：意味的 PoW の定式化

HEB は，人間が本来持っていた「煩雑さ」という安全装置を，エージェントに対する「推論コストの要求（Semantic PoW）」として再構築する．図 3 にそのトランザクション・シーケンスを示す．

(1) 状態再構築と文脈依存の正当化要求

破壊的な操作を要求する際，エージェントは自身の行動履歴 H_t に基づく構造化された「Progress Receipt (PR)」[1] を生成しなければならない．HEB は，エージェントに対し文脈に基づく意味的推論パズル Q を提示し，その解答 $\pi \leftarrow \mathrm{LLM}(Q \cup H_t)$ を要求する．コンテキストが汚染されている場合，正しい解答は生成できず，推論リソースを浪費するだけの「摩擦」として機能する．

(2) ハードゲートによる決定論的検証

HEB 内に一時的に設置されるプロキシ「Policy Enforcer (PE)」[1] は，提出された PR に対し，非決定的な推論を一切用いず，ステートレスかつ決定論的な「ハードゲート」検証を行う [1]．

$$\mathrm{Verify}(\pi, A) = \begin{cases} 1 & (\mathrm{ValidSig} \wedge \mathrm{HashChainIntact}) \\ 0 & (\mathrm{otherwise}) \end{cases} \tag{1}$$

署名不正やセッション束縛の違反など，明白な違反を即座に弾くことで，エージェントの非決定的な暴走を安全な状態遷移の枠内に抑え込む．

(3) 制約連鎖型トークンによる漸進的認可

エージェントは最初から高権限を得ることはできず，タスクの正常完了という証拠を積み重ねる必要がある [1]．検証結果は重み付き幾何平均 v_i として集約され，トラスト T_i が更新される [1]．

$$v_i = \prod_{j=1}^{k} (w_i^{(j)})^{\beta_j} \tag{2}$$

$$T_i = T_{i-1} + \alpha_i v_i (1 - T_{i-1}) \tag{3}$$

このトラスト値が，タスクの残余リスク R_{eff} の許容上限を満たした場合（$R_{eff} \leq g(T_{dec})$）にのみ，次ステップの Capability Token（CT）が連鎖的に発行される [1]．このハッシュ連鎖こそが，攻撃者にとって突破困難な認知的摩擦である．

6.3 決定論的実行による「検証可能な自律性」の確立

HEB は，エージェントの非決定的な意図を，決定論的なポリシーと残余リスクの数学的評価 [1] に変換するコンパイラとして機能する．これにより，既存のパッチ的防御では防げなかった「文脈破綻を伴う論理的攻撃」を構造的に無効化する．

6.4 実装可能性と進行中のシステム検証（On-going Work）

新規のセキュリティ機構導入における最大の障壁は，既存のバックエンドシステムや API 群に対する大規模な改修コストである．本提案における Hard Execution Boundary（HEB）および意味的 PoW の枠組みは，マイクロサービスアーキテクチャにおける**サイドカー・プロキシ・パターン（Sidecar Proxy Pattern）**を応用することで，既存のシステムコードを一切変更することなく実現可能である．

図 4 に，サイドカー・プロキシ・パターンを用いた HEB の配置モデルを示す．現在，著者らは既存の API ゲート

© 2026 Information Processing Society of Japan

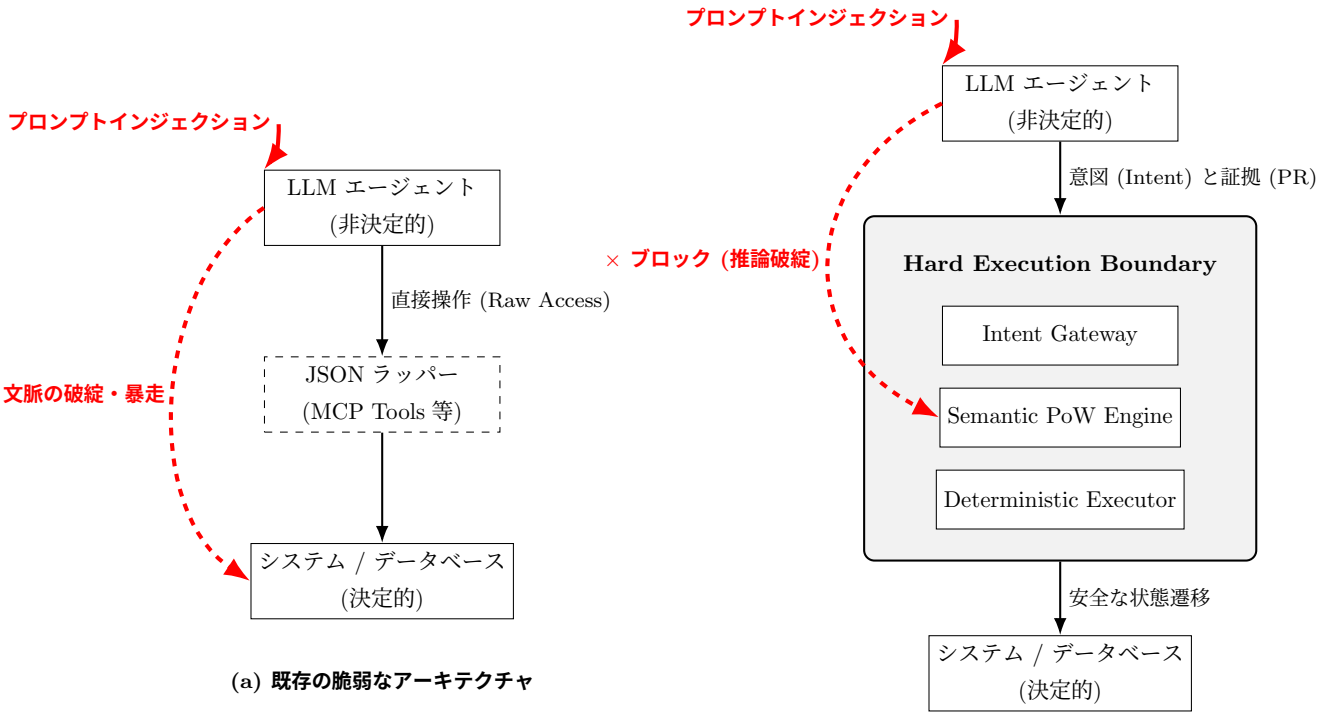


図 2 既存の JSON ラッパーと提案する HEB の比較．HEB は認知的摩擦（Semantic PoW）を用いて非決定的な暴走を決定論的に遮断する．

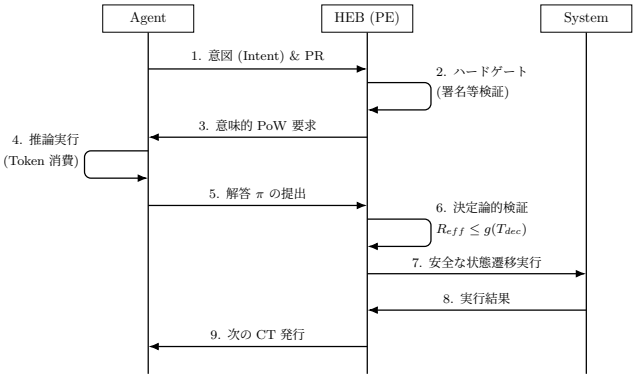


図 3 意味的 PoW のトランザクション・シーケンス．非決定的な推論コストを強要しつつ，検証は決定論的に行う．

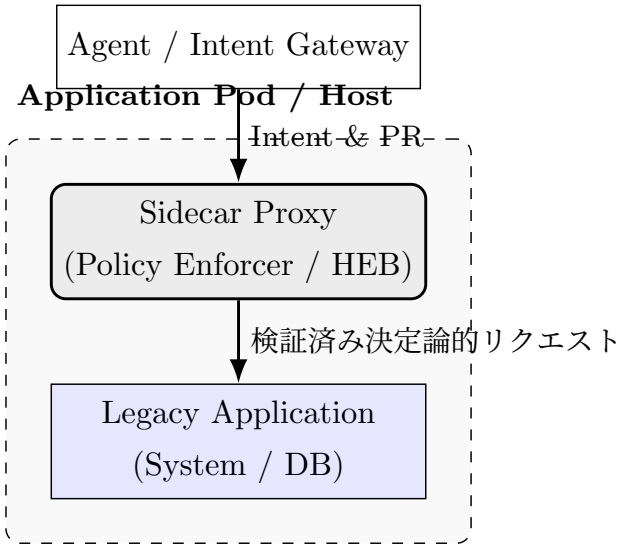


図 4 サイドカー・プロキシ・パターンによる HEB の配置．

ウェイやリバースプロキシ（Envoy 等）の拡張モジュールとして，ステートレスな Policy Enforcer（PE）を配置するプロトタイプ実装を進めている（on-going work）．エージェントからのリクエストはすべてこのサイドカープロキシを通過し，JSON ペイロードの署名検証，ハッシュ連鎖の整合性チェック，および Semantic PoW のパズル提示・検証が処理される．バックエンドのレガシーアプリケーションやデータベースは，プロキシから転送される「検証済みの決定論的リクエスト」のみを受け付けるため，既存のビジネスロジック層を一切改修することなく漸進的（Incremental）に HEB 環境へと移行できる．また，意味的 PoW による追加の推論オーバーヘッド（1 回あたり約 0.01 USD，1.5〜2.0 秒）は，トランザクションの重要度（残余リスク R_{eff}）に応じて動的に調整可能であり，セキュリティとパフォーマンスのトレードオフを制御可能にする実証実験を現在進行形で進めている．本発表（登壇時）には，これらのプロトタイプにおけるストレステストおよび攻撃耐性の実証データを示す予定である．

7. おわりに

本稿では，Agentic AI と実社会を繋ぐ既存のミドルウェ

© 2026 Information Processing Society of Japan

アが抱える構造的脆弱性について論じ，次世代システム基盤のアーキテクチャ要件を提示した．既存の人間用インターフェースを単に JSON でラップするアプローチは，人間が本来持っていた「認知的摩擦」という安全装置をシステムから剥奪するものである．瞬時に悪意ある主体へと変貌し得る非決定的なエージェント [1] に対し，決定的な状態遷移権限を与え続ければ，被害の規模と速度は計り知れない．

この打開策として，本稿ではシステム基盤の最底辺に「Hard Execution Boundary（HEB）」を設けることを提案した．HEB は，エージェントに対し文脈の正当化要求といった「意味的な Proof of Work（Semantic PoW）」を課すことで，認知的摩擦をシステム構造として再構築する．この境界は，亀谷ら [1] が提案するハードゲートと制約連鎖型トークン（MPT）によって統制されることで，非決定的な暴走を封じ込める．さらに，サイドカープロキシを通じた実装可能性と，現在進行形（on-going work）で進めているプロトタイプ検証の枠組みを示すことで，本アーキテクチャが実務的なシステムインフラへ直ちに適用可能であることを論じた．登壇時には，実装に基づく具体的なオーバーヘッドと防御効果の実証データをお見せする予定である．

人間と AI の共生社会を実現するための「Human-in-Command」原則を単なる倫理のスローガンで終わらせないためには，法的証拠能力と物理的抑止力を伴うハードなインフラの設計が不可欠である．今後は，本アーキテクチャの大規模なストレステスト環境での実証評価や，複雑系ネットワークにおけるマクロな経済リスク（合成の誤謬）を防ぐための自律的サーキットブレーカーの設計へと研究を展開していく．

参考文献

[1] 亀谷広務, 八槇博史. 無記憶漸進的トラスト: Agentic ai のための非集権的トラストアーキテクチャー. コンピュータセキュリティシンポジウム 2026 (CSS 2026), 2026. （本大会同時投稿）.

[2] Stephen P. Marsh. *Formalising Trust as a Computational Concept*. PhD thesis, University of Stirling, 1994.

[3] Cristiano Castelfranchi and Rino Falcone. Principles of trust for mas: Cognitive anatomy, social importance, and quantification. In *Proceedings of the International Conference on Multi Agent Systems (ICMAS)*, pp. 72–79, 1998.

[4] Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina. The eigentrust algorithm for reputation management in p2p networks. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pp. 640–651, 2003.

[5] William H. Winsborough, Kent E. Seamons, and V. E. Jones. Automated trust negotiation. In *Proceedings of the DARPA Information Survivability Conference and Exposition (DISCEX)*, 2000.

[6] Kai Greshake, Sahar Abdelabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[7] Ben Nassi, et al. Compromptmized: Unleashing zero-click worms that target genai-powered applications. *arXiv preprint*.

[8] L. Yan, et al. Trust no tool: Evaluating and defending llm agents under untrusted tool feedback. *arXiv preprint*.

[9] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io/`, 2025.

[10] 八槇博史, ほか. Model context protocol エコシステムのセキュリティ分析と mcp fuzzer の開発. 情報処理学会全国大会講演論文集, 2026.

[11] A. Somoza. Agent trust negotiation: Capability, delegation, and provenance binding for ai agents. Internet-Draft, IETF, 2026.

[12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. Zero trust architecture. Technical Report NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[13] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think. In *Proceedings of the Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Y. Chen, et al. Reasoning models don't always say what they think. *arXiv preprint*, 2025.

© 2026 Information Processing Society of Japan